Computational tools for the projects of the course Finančni praktikum 2026/2027*

Riste Škrekovski & Andrej Kolar-Požun

October 26, 2026

Contents

1 Introduction **2**

2 SageMath **2**
2.1 Getting started . . . 2
2.2 Sage is (almost) Python . . . 3
2.3 Basic graph related Sage commands . . . 3

3 Generating graphs: `geng` and other generators **4**

4 Some more tips **5**
4.1 Planar graphs . . . 5
4.2 Multigraphs in SageMath . . . 6
4.3 Numerical precision and ties . . . 7
4.4 Distances and numbers of shortest paths . . . 7

5 Exact computation **8**
5.1 Backtracking . . . 8
5.2 Integer linear programming in SageMath . . . 9
5.2.1 Basic commands . . . 9
5.2.2 Example: independence number . . . 9
5.2.3 Example: rich edge-colorings . . . 10
5.3 SAT solvers in SageMath . . . 10

6 Heuristic search **12**
6.1 The space of graphs . . . 12
6.2 Random walk . . . 13
6.3 Hill climbing . . . 13
6.4 Simulated annealing . . . 14
6.5 Other metaheuristics . . . 15

*Department of Mathematics, FMF, University of Ljubljana, Slovenia.

7 Long computations 16

8 Using AI tools 16

9 What to report 16

1 Introduction

Almost every project of this course has the same structure. We are given a graph parameter and a conjecture (or a question) about it, and we want to collect computational evidence for or against it. This is usually done in two steps:

1. a *systematic search*: we check *all* graphs of a given type up to some (small) order n. The result is exact, e.g. "the conjecture holds for all connected subcubic graphs on at most 12 vertices"

2. a *heuristic search* over larger graphs, where a systematic search is no longer possible. We cannot check all graphs, so we try to find the "most promising" ones (e.g. the ones closest to a counterexample).

For both steps one needs a correct and reasonably fast program that computes the parameter for a given graph. This document briefly describes the tools we recommend: SageMath (Section 2), graph generators (Section 3), exact methods – backtracking, integer linear programming and SAT solvers (Sections 5.1, 5.2 and 5.3), heuristic methods – random walks, hill climbing and simulated annealing (Section 6), multigraphs (Section 4.2), long computations (Section 7), AI tools (Section 8), and what we expect in the report (Section 9).

The code in this document is written in SageMath.

2 SageMath

SageMath (`https://www.sagemath.org`) is a free mathematical system based on Python. It contains a large library for graphs (class `Graph`), many graph generators, the nauty programs (Section 3), and an interface to integer linear programming solvers (Section 5.2).

2.1 Getting started

There are three common ways to use SageMath:

- *online*, without installation, at SageMathCell (`https://sagecell.sagemath.org`). Good for first experiments, but slow for long computations,
- *locally*, installed on your computer (see `https://doc.sagemath.org/html/en/installation/`) either in a Jupyter notebook (`sage -n jupyter`) or in the terminal (`sage`),
- *from the command line*: `sage script.sage` or `sage script.py`. This is the right way for long computations (Section 7).

2.2 Sage is (almost) Python

SageMath is Python with a preprocessor, which causes a few surprises:

- `2^10` is the power 2^{10} in Sage, but the bitwise XOR in Python (For powers, just use `2**10` which works both in Sage and Python),
- in a `.sage` file, the number `1` is a Sage `Integer`, not a Python `int` so some Python functions do not accept it, e.g. `random.seed(1)` fails and one has to write `random.seed(int(1))`,
- `1/3` is the exact rational number 1/3, not 0.333...,
- a `.py` file is pure Python – to use Sage in it, start it with `from sage.all import *`.

2.3 Basic graph related Sage commands

Documentation of the graph library: https://doc.sagemath.org/html/en/reference/graphs/. In a notebook, `G.diameter?` shows the documentation of a method, and `G.<TAB>` lists all methods.

Creating a graph can be done in multiple ways:

```
G = graphs.PetersenGraph()          # a named graph
G = Graph([(0,1), (1,2), (2,0)])    # a graph given by its edges
G = Graph('D]w')                    # a graph given in graph6 format
G.order(), G.size(), G.degree()     # number of vertices, edges, degrees
G.vertices(sort=True), G.edges(sort=True, labels=False)
G.neighbors(0), G.edges_incident(0) # neighbours and edges at a vertex
G.add_edge(0, 3); G.delete_edge(0, 1); G.delete_vertex(2)
H = G.copy()                        # always copy before modifying!
```

Some of the many graph invariants and tests:

```
G.is_connected(), G.is_planar(), G.is_bipartite(), G.is_tree()
G.diameter(), G.radius(), G.girth(), G.distance(0, 5)
G.chromatic_number(), G.chromatic_index()
G.independent_set(), G.clique_number(), G.matching()
G.automorphism_group().order()
G.is_isomorphic(H)                  # isomorphism test
G.canonical_label().graph6_string() # the same string for isomorphic graphs
```

The canonical label is useful if you want to store only non-isomorphic graphs (e.g. all extremal graphs found by a heuristic): put their canonical graph6 strings into a Python `set`.

For eigenvalues we have `G.spectrum()`, which computes *exact* algebraic numbers, which is very slow. For numerical computations (e.g. with many graphs) use `numpy`:

```
import numpy
ev = numpy.linalg.eigvalsh(G.adjacency_matrix().numpy())   # sorted, floats
```

Named graphs, families and random graphs (and many more):

```
graphs.CompleteGraph(n), graphs.CycleGraph(n), graphs.PathGraph(n)
graphs.CompleteBipartiteGraph(m, n), graphs.CubeGraph(d)
graphs.GeneralizedPetersenGraph(n, k), graphs.CirculantGraph(n, [1, 3])
graphs.RandomGNP(n, p), graphs.RandomRegular(d, n), graphs.RandomTree(n)
```

Drawing and files:

```
G.show()                            # draw the graph (in a notebook)
G.plot().save('G.png')              # save a picture
with open('graphs.g6', 'a') as F:   # append a graph to a file
    F.write(G.graph6_string() + '\n')
```

3 Generating graphs: `geng` and other generators

The program `geng` from the package *nauty* (`https://pallini.di.uniroma1.it`) generates all non-isomorphic graphs of a given order, one graph per line in graph6 format. It is included in SageMath, and it can also be used on its own (on Linux, e.g. `apt install nauty`, on a Mac, `brew install nauty`). Useful options:

`-c`	connected graphs only
`-d#`, `-D#`	lower / upper bound on the degrees
`-t`, `-f`	triangle-free / 4-cycle-free graphs
`-tf`	both, i.e. girth at least 5
`-b`	bipartite graphs
`-q`	suppress auxiliary output
`-u`	do not output the graphs, only count them
`n mine:maxe`	only graphs with between `mine` and `maxe` edges
`res/mod`	split the job into `mod` parts and output part `res`

For example, `geng -c -D3 12 > g12.g6` writes all connected subcubic graphs of order 12 into a file, `geng -c -d3 -D3 12` generates the connected cubic graphs, and `geng -c 10 9:9` the trees of order 10. `geng -c -d3 -D3 -tf 18` gives the connected cubic graphs of order 18 and girth at least 5 (do not use the filter `pickg -g5` for this: it keeps the graphs of girth *exactly* 5). In SageMath one can loop over the graphs directly:

```
for G in graphs.nauty_geng("-c -D3 10"):
    ...                                 # test G
```

or read a file line by line with `G = Graph(line.strip())`.

The number of graphs grows very fast with the order. For example, the numbers of connected subcubic graphs are:

n	4	5	6	7	8	9	10	11	12	13	14
graphs	6	10	29	64	194	531	1733	5524	19430	69322	262044

For all connected graphs (without a degree bound) the numbers grow much faster: there are 11 117 of order 8, 261 080 of order 9 and 11 716 571 of order 10. So first test how long your program needs for small orders and estimate how far you can go. A larger order can be split into parts that are processed in parallel: `geng -c -D3 14 0/8`, ..., `geng -c -D3 14 7/8` together give all graphs of order 14, each exactly once.

For special classes there are faster specialised generators: `genreg` (regular graphs), `plantri` (planar graphs), `buckygen` (fullerenes), `snarkhunter` (cubic graphs and snarks), `genbg` (bipartite graphs, part of nauty) and `multig` (multigraphs, part of nauty, Section 4.2). Lists of many interesting graphs (e.g. all snarks, cubic graphs of given girth, fullerenes) can be downloaded from the House of Graphs (https://houseofgraphs.org). Notice that `genreg` and `snarkhunter` are not included in SageMath and have to be installed separately. Regular graphs can also be generated (more slowly) by `geng -c -d# -D#`. Some families are not built in, but are easy to construct, e.g. the Möbius ladder on $2n$ vertices is `graphs.CirculantGraph(2*n, [1, n])`.

4 Some more tips

4.1 Planar graphs

For small orders, the simplest way is to generate all graphs with `geng` and keep the planar ones:

```
for G in graphs.nauty_geng("-c 8"):
    if G.is_planar():
        ...                              # test G
```

This is easy, but slow for larger orders, since most graphs are not planar: there are 1 052 805 connected planar graphs of order 10 (out of 11 716 571 connected graphs), and filtering them takes several minutes.

A much faster way is the program `plantri` by Brinkmann and McKay (https://users.cecs.anu.edu.au/~bdm/plantri/), which generates planar graphs directly. It is included in SageMath:

```
graphs.planar_graphs(n, minimum_connectivity=1)  # connected planar graphs
graphs.planar_graphs(n, minimum_connectivity=3)  # 3-connected planar graphs
graphs.planar_graphs(n, minimum_degree=3)        # with minimum degree >= 3
graphs.triangulations(n)                         # planar triangulations
graphs.quadrangulations(n)                       # planar quadrangulations
graphs.fullerenes(n)                             # fullerenes (by buckygen)
```

For example, there are 257 3-connected planar graphs and 14 triangulations of order 8, and 2 fullerenes on 28 vertices. Always give the connectivity (or the minimum degree) explicitly. Even the 3-connected planar graphs of order 13 (about 10^8 graphs) can be processed in this way. If the test of a graph is expensive, first apply a fast filter (e.g. a greedy coloring) and use an exact method only for the graphs which do not pass it.

Warning. `plantri` generates *plane* graphs, i.e. planar graphs together with an embedding in the plane. A 3-connected planar graph has essentially only one embedding, but a planar graph of connectivity 1 or 2 may have several, and then it is output several times. For example, `graphs.planar_graphs(6, minimum_connectivity=1)` gives 179 plane graphs, but only 99 non-isomorphic graphs. If you need every graph only once, remove the duplicates:

```
seen = set()
for G in graphs.planar_graphs(n, minimum_connectivity=1):
    s = G.canonical_label().graph6_string()
    if s not in seen:
        seen.add(s)
        ...                              # test G
```

(For $n \geq 11$ the set `seen` becomes large. In this case it is better to test each plane graph, even if some graphs are tested more than once.) The embedding is available as `G.get_embedding()`, and the faces as `G.faces()`, which is useful e.g. for problems about faces or cycles of given lengths.

4.2 Multigraphs in SageMath

SageMath supports (loopless) multigraphs: it is enough to allow multiple edges when creating the graph. Give every edge a different label, so that parallel edges can be distinguished (e.g. in order to give them different colors):

```
G = Graph([(0,1,'a'), (0,1,'b'), (1,2,'c')], multiedges=True)
G.edges(sort=True)          # [(0,1,'a'), (0,1,'b'), (1,2,'c')]
G.degree()                  # [2, 3, 1]
G.has_multiple_edges()      # True
G.multiple_edges(sort=True) # the parallel edges
G.delete_edge(0, 1, 'a')    # delete one particular edge
```

Many basic methods (degrees, neighbours, edges, connectivity) work for multigraphs as well, but some algorithms do not. For example, `edge_coloring()` or `line_graph()` stop with the error "*This method is not known to work on graphs with multiedges*". In such a case, write the algorithm yourself, or use `G.allow_multiple_edges(False)` on a copy of the graph when only the underlying simple graph is needed. Also be careful when you write your own functions: e.g. when you list the edges adjacent to an edge $e = uv$, a parallel edge is incident with both u and v, but it should be counted only once.

All multigraphs can be generated with the program `multig` from nauty, which replaces the edges of the input graphs by multiple edges in all possible ways. For example,

```
geng -c -D3 6 | multig -m3 -D3 -T
```

gives all connected loopless multigraphs of order 6 with maximum degree at most 3 and edge multiplicity at most 3. With option `-T`, every line has the form `n m v1 w1 k1 v2 w2 k2 ...`, i.e. the number of vertices and of (distinct) edges, followed by the edges with their multiplicities. Such a line is read into SageMath as follows:

```
t = [int(s) for s in line.split()]
n, m = t[0], t[1]
E = []
for i in range(m):
    u, v, k = t[2+3*i : 5+3*i]
    E += [(u, v)] * k
G = Graph([(u, v, i) for i, (u, v) in enumerate(E)], multiedges=True)
```

For a heuristic search over larger multigraphs, random multigraphs of maximum degree at most Δ can be obtained from a simple graph (e.g. of maximum degree at most $\Delta - 1$) by adding random parallel edges:

```
from random import choice
def add_parallel_edges(G, Delta, tries=100):
    E = G.edges(labels=False, sort=True)
    M = Graph([(u, v, i) for i, (u, v) in enumerate(E)], multiedges=True)
```

```
    for _ in range(tries):
        u, v, _ = choice(list(M.edges(sort=False)))
        if M.degree(u) < Delta and M.degree(v) < Delta:
            M.add_edge(u, v, M.size())    # new parallel edge, new label
    return M
```

4.3 Numerical precision and ties

Dealing with floating point numbers often results in rounding errors, so never test them for equality with `==`. Instead compare with a tolerance, e.g. `abs(a - b) < 1e-9`, and report how many graphs are (almost) tied.

If the values differ by many orders of magnitude (e.g. $\sum_{uv}(d(u)^2 + d(v)^2)^\alpha$ for $\alpha = -50$), use higher precision, e.g. `R = RealField(200)` and `R(x)`, or exact arithmetic.

Often non-isomorphic graphs have *exactly* the same value because the value depends only on some simple data (e.g. a degree-based index depends only on the multiset of the pairs of degrees at the edges). In this case group the graphs by this data first – this finds the exact ties without any rounding errors.

4.4 Distances and numbers of shortest paths

Do not enumerate all paths if you need only shortest paths. One breadth-first search from a vertex u gives the distances $d(u,v)$ and the numbers $\sigma(u,v)$ of shortest u,v-paths for all v: the number of shortest paths to w is the sum of these numbers over the neighbours of w one step closer to u. Hence, e.g., the total geodesic distance $\sum_{\{u,v\}} d(u,v)\,\sigma(u,v)$ and the number of all geodesics are computed in time $O(nm)$:

```
def geodesic_counts(G, u):
    dist = {u: 0}; sigma = {u: 1}; queue = [u]
    for v in queue:                       # the queue grows during the loop
        for w in G.neighbor_iterator(v):
            if w not in dist:
                dist[w] = dist[v] + 1; sigma[w] = 0; queue.append(w)
            if dist[w] == dist[v] + 1:
                sigma[w] += sigma[v]      # shortest paths to w through v
    return dist, sigma

def tgd(G):                               # total geodesic distance
    total = 0
    for u in G:
        dist, sigma = geodesic_counts(G, u)
        total += sum(dist[v] * sigma[v] for v in G)
    return total // 2                     # every pair was counted twice
```

For example, `tgd(graphs.CycleGraph(6))` returns 36.

5 Exact computation

5.1 Backtracking

Many parameters are defined through a "best" set, partition or coloring, e.g. the smallest resolving set, or the smallest number of colors of a coloring with some property. The naive approach – try all possibilities – is hopeless already for small graphs ($5^{15} \approx 3 \cdot 10^{10}$ colorings of 15 edges with 5 colors). A *backtracking* algorithm builds the solution step by step (e.g. colors the edges one by one) and *goes back* as soon as the partial solution cannot be completed. Some advice:

- *Prune early*: after each step check all conditions that can already be violated (e.g. two adjacent edges of the same color).
- *Choose a good order*: e.g. color the edges in the order of a breadth-first search, so that conflicts are detected soon.
- *Break symmetries*: the names of the colors do not matter, so the first edge may get color 1, the next new color is always the smallest unused one, etc. This alone speeds the search up by a factor of up to $k!$.
- *Test it*: compare the results with a naive (slow) program on all very small graphs, and on graphs for which the answer is known.

Example: edge-colorings. The following function finds a proper k-edge-coloring of a connected graph G, or returns `None` if there is none. The edges are colored in the order of a breadth-first search (a good order). The function `extend(i, m)` colors the edges $E[i], E[i+1], \ldots$ one by one, assuming that $E[0], \ldots, E[i-1]$ are already colored with the colors $0, \ldots, m-1$. For the edge $E[i]$ it tries the colors one after another, from the smallest, and skips every color which already appears at one of its ends (pruning). It also uses at most one new color, i.e. it tries only the colors $0, \ldots, m$ (symmetry breaking). If no color works, it returns `False`, and we go back to the previous edge, which tries its next color (backtracking):

```
def edge_coloring(G, k):
    order = G.breadth_first_search(G.vertices(sort=False)[0])
    pos = {v: i for i, v in enumerate(order)}
    E = sorted(G.edges(labels=False, sort=False),
               key=lambda e: sorted((pos[e[0]], pos[e[1]])))
    col = {}                              # the partial coloring
    used = {v: set() for v in G}          # the colors at each vertex
    def extend(i, m):                     # color E[i], E[i+1], ...; m colors used
        if i == len(E):
            return True
        u, v = E[i]
        for c in range(min(m + 1, k)):  # symmetry: at most one new color
            if c not in used[u] and c not in used[v]:     # prune: keep it proper
                col[u, v] = c; used[u].add(c); used[v].add(c)
                if extend(i + 1, max(m, c + 1)):
                    return True
                del col[u, v]; used[u].remove(c); used[v].remove(c)  # go back
        return False
    return col if extend(0, 0) else None
```

The chromatic index $\chi'(G)$ is the smallest k for which a coloring is found. For example, `edge_coloring(graphs.PetersenGraph(), 3)` returns `None` and with $k = 4$ it returns a coloring, both in a few milliseconds. To test the function, compare it with `G.chromatic_index()` on all small graphs:

```
for G in graphs.nauty_geng("-c 7"):
    k = min(k for k in range(1, 8) if edge_coloring(G, k) is not None)
    assert k == G.chromatic_index()
```

The same scheme works for other colorings: only the test "can edge $E[i]$ get color c?" changes, and possibly a final test when all edges are colored (e.g. for a rich coloring, that every vertex is incident with a rich edge).

For many problems it is easier to write an integer linear program instead.

5.2 Integer linear programming in SageMath

Many graph parameters are defined as the minimum or maximum of something over all sets or colorings, and they can be written as an *integer linear program* (ILP): we introduce integer (usually binary, i.e. 0/1) variables, linear constraints, and a linear objective function. SageMath then calls an ILP solver (by default GLPK, faster solvers such as Gurobi or CPLEX can be used if installed, e.g. `MixedIntegerLinearProgram(solver="Gurobi")`. Both are free for students). For small and medium graphs this is often the fastest way to compute a parameter exactly, and it is much easier to program correctly than a clever backtracking.

5.2.1 Basic commands

```
p = MixedIntegerLinearProgram(maximization=False)
x = p.new_variable(binary=True)      # x[...] are 0/1 variables
p.add_constraint(x[0] + x[1] >= 1)   # a linear constraint
p.set_objective(x[0] + x[1])         # the objective function
p.solve()                            # the optimal value
p.get_values(x)                      # the values of the variables
```

The variables can be indexed by anything, e.g. `x[v]` for a vertex v, or `x[e, c]` for an edge e and a color c. Use `integer=True` instead of `binary=True` for general integer variables. If the problem has no solution, `p.solve()` raises a `MIPSolverException`.

5.2.2 Example: independence number

The independence number $\alpha(G)$ is the maximum of $\sum_v x_v$ over binary x_v with $x_u + x_v \le 1$ for every edge uv:

```
def independence_number(G):
    p = MixedIntegerLinearProgram(maximization=True)
    x = p.new_variable(binary=True)
    for u, v in G.edges(labels=False, sort=False):
        p.add_constraint(x[u] + x[v] <= 1)
    p.set_objective(sum(x[v] for v in G))
    return p.solve()
```

(Sage already has `G.independent_set()`, the example only shows the method.)

5.2.3 Example: rich edge-colorings

A rich k-edge-coloring of a connected G (with at least 2 vertices) exists if and only if the following system has a solution in binary variables $x_{e,c}$ ($x_{e,c} = 1$ if and only if the edge e has color c) and r_e ($r_e = 1$ if e is required to be rich). Here $N(e)$ denotes the set of edges adjacent to e.

$$\begin{aligned}
\textstyle\sum_{c=1}^{k} x_{e,c} &= 1 && \text{for every edge } e,\\
\textstyle\sum_{e\ni v} x_{e,c} &\le 1 && \text{for every vertex } v \text{ and color } c,\\
\textstyle\sum_{f\in N(e)} x_{f,c} &\le 1 + |N(e)|\,(1-r_e) && \text{for every edge } e \text{ and color } c,\\
\textstyle\sum_{e\ni v} r_e &\ge 1 && \text{for every non-isolated vertex } v.
\end{aligned}$$

The first two lines say that we have a proper edge-coloring. If $r_e = 1$, the third line says that the edges adjacent to e have pairwise distinct colors, i.e. e is rich; if $r_e = 0$, it says nothing (this is the usual "big-M" trick for switching a constraint on and off). The last line says that every vertex is incident with a rich edge. In SageMath:

```
from sage.numerical.mip import MIPSolverException

def rich_coloring(G, k):
    E = G.edges(labels=False, sort=True)
    p = MixedIntegerLinearProgram(maximization=False)
    x = p.new_variable(binary=True)
    r = p.new_variable(binary=True)
    for e in E:
        p.add_constraint(sum(x[e, c] for c in range(k)) == 1)
    for v in G:
        Ev = [e for e in E if v in e]
        for c in range(k):
            p.add_constraint(sum(x[e, c] for e in Ev) <= 1)
        p.add_constraint(sum(r[e] for e in Ev) >= 1)
    for e in E:
        N = [f for f in E if f != e and set(f) & set(e)]
        for c in range(k):
            p.add_constraint(sum(x[f, c] for f in N)
                             <= 1 + len(N) * (1 - r[e]))
    try:
        p.solve()
    except MIPSolverException:
        return None                     # no rich k-edge-coloring
    X = p.get_values(x)
    return {e: c for e in E for c in range(k) if X[e, c] > 0.5}
```

Then $\chi'_r(G)$ is the smallest k for which `rich_coloring(G, k)` does not return `None`; for example, it returns `None` for $G = K_4$ and $k = 4$, and a coloring for $k = 5$. Symmetric solutions (permuting the colors) slow the solver down – it helps to fix the colors of the edges at one vertex in advance. Always check the solution returned by the solver with an independent (simple) function.

5.3 SAT solvers in SageMath

Many coloring questions ("does G have a coloring with k colors such that ...?") are decision problems which can be written as a Boolean formula in conjunctive normal form and solved by

a *SAT solver*. For such problems a SAT solver is often much faster than an ILP: for the 25-vertex graph of the conflict-free coloring project, an ILP (solved by GLPK) needs about two minutes to show that there is no conflict-free 3-coloring, while the SAT solver PicoSAT with the code below needs 0.01 seconds.

In SageMath, variables are positive integers $1, 2, 3, \ldots$, a negative number denotes the negation, and a clause is a tuple of such numbers (their disjunction). The PicoSAT interface needs the Python package `pycosat`. If `SAT(solver="picosat")` fails, install it with `sage -pip install pycosat`:

```
from sage.sat.solvers.satsolver import SAT
S = SAT(solver="picosat")    # needs the package pycosat
S.add_clause((1, 2))         # x1 or x2
S.add_clause((-1,))          # not x1
S()                          # (None, False, True): x1 = False, x2 = True
                             # the answer is False if there is no solution
```

Typical constraints: "vertex v gets at least one color" is one clause $(x_{v,1} \vee \cdots \vee x_{v,k})$; "at most one color" are the clauses $(\neg x_{v,c} \vee \neg x_{v,d})$ for all $c < d$; an implication $y \Rightarrow (a \vee b)$ is the clause $(\neg y \vee a \vee b)$.

Example: conflict-free colorings. A coloring is conflict-free (with respect to closed neighborhoods) if every vertex has a color which appears exactly once in $N[v]$. We use variables $x_{v,c}$ (vertex v has color c) and $y_{v,c}$ (color c appears exactly once in $N[v]$):

```
import itertools
from sage.sat.solvers.satsolver import SAT

def conflict_free_coloring_exists(G, k):
    var = {}
    def x(*key):                          # number the variables 1, 2, 3, ...
        if key not in var: var[key] = len(var) + 1
        return var[key]
    S = SAT(solver="picosat")
    for v in G:
        S.add_clause(tuple(x('x', v, c) for c in range(k)))
        for c, d in itertools.combinations(range(k), 2):
            S.add_clause((-x('x', v, c), -x('x', v, d)))
    for v in G:
        N = [v] + G.neighbors(v)
        S.add_clause(tuple(x('y', v, c) for c in range(k)))
        for c in range(k):
            y = x('y', v, c)
            S.add_clause((-y,) + tuple(x('x', u, c) for u in N))
            for u, w in itertools.combinations(N, 2):
                S.add_clause((-y, -x('x', u, c), -x('x', w, c)))
    return S() is not False
```

Symmetry breaking. The names of the colors do not matter, so a solver may waste time on equivalent colorings. Fix in advance what can be fixed: the color of one vertex, or the colors of all edges at a vertex of maximum degree (they are pairwise distinct in a proper edge-coloring). For rich edge-colorings of K_5, for example, the solver does not finish within minutes without

this, and needs a fraction of a second with it. With such tricks SAT solvers handle even graphs with hundreds of vertices.

Counting constraints. A constraint such as "at most t of the variables $y_1, \dots, y_m$ are true" (e.g. at most t vertices without a unique color) can be encoded with the module `pysat.card` of `python-sat` (`CardEnc.atmost`; for very small t, one may simply add the clause $(\neg y_{i_0} \vee \dots \vee \neg y_{i_t})$ for every set of $t+1$ indices).

Faster solvers can be installed: `sage -pip install pycryptosat` or `sage -i kissat` (then use `SAT(solver="cryptominisat")` or `SAT(solver="kissat")`), or `sage -pip install python-sat`, which gives access to many modern solvers (e.g. CaDiCaL, Glucose) from Python. As always, check the colorings found by the solver with an independent function.

6 Heuristic search

For larger graphs a systematic search is not possible, so we search heuristically [2, 1]. Heuristics do not prove anything, but they are very good at finding counterexamples and extremal graphs, which can then be verified exactly.

6.1 The space of graphs

We think of the set of all graphs of a given type (e.g. connected graphs of order n with maximum degree at most Δ) as a *space*, in which two graphs are *neighbours* if one is obtained from the other by a small change, for example:

- add an edge or delete an edge;
- move an edge, i.e. delete an edge uv and add an edge uw;
- *switch* two edges: replace edges ab and cd by ac and bd (where a, b, c, d are distinct and ac, bd are not edges, so that the graph stays simple; a switch keeps all degrees, so it is suitable e.g. for cubic graphs);
- for trees: delete an edge and reconnect the two components by another edge.

After every change we check that the new graph is still in our class (connected, maximum degree at most Δ, ...); if not, we reject it. The same idea works for other search spaces, e.g. the colorings of a fixed graph, where a neighbour is obtained by recoloring one edge. For example, a switch which keeps the graph connected; the edge cd is reversed with probability 1/2, so that both possible switches (ac, bd and ad, bc) are tried:

```
from random import sample, random
def switch_edges(G):
    H = G.copy()
    E = list(H.edges(labels=False, sort=False))
    (a, b), (c, d) = sample(E, 2)
    if random() < 0.5:
        c, d = d, c             # the other switch
    if (len({a, b, c, d}) == 4 and not H.has_edge(a, c)
            and not H.has_edge(b, d)):
```

```
        H.delete_edges([(a, b), (c, d)])
        H.add_edges([(a, c), (b, d)])
        if H.is_connected():
            return H
    return G                    # the move is rejected
```

6.2 Random walk

A *random walk* starts at some graph and repeatedly moves to a random neighbour. It does not use any objective function; it just visits many different graphs, and we test each of them (e.g. whether it is a counterexample). Random walks are very easy to program and are useful for testing a conjecture on "typical" larger graphs. In SageMath one step of a random walk can look as follows:

```
def random_neighbour(G, Delta):
    H = G.copy()
    u, v = sample(H.vertices(sort=False), 2)
    if H.has_edge(u, v):
        H.delete_edge(u, v)
    elif H.degree(u) < Delta and H.degree(v) < Delta:
        H.add_edge(u, v)
    return H if H.is_connected() else G

G = graphs.CycleGraph(20)
for step in range(10**6):
    G = random_neighbour(G, 3)
    ...                                 # test G
```

Another simple possibility is to test many *random graphs* (`graphs.RandomGNP`, `graphs.RandomRegular`, ...); but notice that random graphs are "typical", while counterexamples are often very special graphs.

6.3 Hill climbing

Hill climbing adds an objective function f that we want to minimize (although the name "hill climbing" comes from maximization) (e.g. the value of an index, or "how far" a graph is from being a counterexample). From the current graph G we move to a random neighbour H only if $f(H) \leq f(G)$ (allowing equality lets the search move along "plateaus"):

```
def hill_climb(G, f, neighbour, steps):
    fG = f(G)
    for _ in range(steps):
        H = neighbour(G)
        fH = f(H)
        if fH <= fG:
            G, fG = H, fH
    return G, fG

# example: a connected subcubic graph on 12 vertices with maximum diameter
G, fG = hill_climb(graphs.CycleGraph(12), lambda H: -H.diameter(),
                   lambda H: random_neighbour(H, 3), 2000)
```

(To maximize a function, minimize its negative.) In this way we quickly reach a *local minimum*, i.e. a graph none of whose neighbours is better. The drawback is that the search stops there, although the global minimum may be elsewhere. Therefore we restart hill climbing many times from different random graphs and keep the best result, or we use simulated annealing, which sometimes accepts also worse moves.

6.4 Simulated annealing

Simulated annealing is a simple and very general local-search method. We need three ingredients:

- a set of *states* x (e.g. graphs of order n, trees with maximum degree Δ, colorings of a fixed graph, ...);
- an *objective function* $f(x)$ that we want to minimize (e.g. the value of an index, or the number of violated conditions);
- a *neighbourhood*: a small random change of a state (e.g. move, add or delete an edge; recolor an edge).

From the current state x we choose a random neighbour y. If y is better, we always accept it. If y is worse, we still accept it with probability $e^{-(f(y)-f(x))/T}$, where $T > 0$ is the *temperature*. At the beginning T is large and the search moves freely; then T slowly decreases and the search more and more behaves as hill climbing. Accepting worse states is what allows the method to escape from local minima.

Algorithm 1: Simulated annealing (minimization of f)

Input: initial state x, initial temperature T_0, final temperature $T_{\min}$, cooling factor $0 < \alpha < 1$, number L of steps at each temperature

$T \leftarrow T_0$; $x_{\text{best}} \leftarrow x$;
while $T > T_{\min}$ *and no solution is found* **do**
  for $i = 1, \dots, L$ **do**
    $y \leftarrow$ random neighbour of x;
    $\delta \leftarrow f(y) - f(x)$;
    if $\delta \le 0$ *or* random$(0,1) < e^{-\delta/T}$ **then**
      $x \leftarrow y$;
      if $f(x) < f(x_{\text{best}})$ **then**
        $x_{\text{best}} \leftarrow x$;
  $T \leftarrow \alpha T$;
return x_{best}

Tuning. Typical values are $\alpha \in [0.95, 0.999]$. Choose T_0 so that at the beginning most of the worse moves are accepted (e.g. T_0 a few times the typical value of δ), and $T_{\min}$ so that at the end almost none is accepted. Slower cooling (larger α or L) gives better results but takes longer. The result depends on luck, so repeat every search several times with different random seeds, and report how often the best value was found. Computing $f(y)$ from scratch in every step is often the slowest part; if possible, compute only the change δ caused by the small modification.

Example: rich edge-colorings. Let the states be all (not necessarily proper) k-edge-colorings of a fixed graph G, let a neighbour be obtained by recoloring one random edge, and let

$$f = (\text{number of pairs of adjacent edges of the same color}) + (\text{number of poor vertices}),$$

where a vertex is *poor* if it is incident with no rich edge. Then $f = 0$ if and only if the coloring is a rich k-edge-coloring.

```
import math
from random import random, choice, randrange

def cost(G, N, col):
    conflicts = sum(1 for e in N for f in N[e] if e < f and col[e] == col[f])
    def rich(e):
        cs = [col[e]] + [col[f] for f in N[e]]
        return len(set(cs)) == len(cs)
    poor = sum(1 for v in G if not any(rich(e)
               for e in G.edges_incident(v, labels=False, sort=True)))
    return conflicts + poor

def anneal_rich(G, k, T0=2.0, alpha=0.999, Tmin=0.01, L=50):
    E = G.edges(labels=False, sort=True)
    N = {e: [f for f in E if f != e and set(f) & set(e)] for e in E}
    col = {e: randrange(k) for e in E}          # random initial coloring
    fx = cost(G, N, col)
    T = T0
    while T > Tmin and fx > 0:
        for _ in range(L):
            e = choice(E); old = col[e]
            col[e] = randrange(k)               # neighbour: recolor one edge
            d = cost(G, N, col) - fx
            if d <= 0 or random() < math.exp(-d / T):
                fx += d                         # accept
            else:
                col[e] = old                    # reject
        T *= alpha
    return fx, col                              # fx == 0: rich coloring found
```

For example, `anneal_rich(graphs.PetersenGraph(), 5)` finds a rich 5-edge-coloring in a few seconds. If we want to search for a counterexample, we can run a second search over graphs: for each graph we run the coloring search several times and prefer graphs for which the smallest value of f found is large. A simple measure of how hard it is to color a graph is the fraction of failed runs of a short annealing. Every counterexample found in this way must then be verified exactly (Sections 5.1, 5.2 and 5.3).

6.5 Other metaheuristics

There are many other metaheuristics [2, 1], e.g. *tabu search* (hill climbing which forbids returning to recently visited states), *genetic algorithms* (a population of states which are combined and mutated), or *variable neighbourhood search*. For our projects, simulated annealing (with restarts) is usually good enough; if you want to try another method, compare it with simulated annealing.

7 Long computations

Long computations (hours or days) should be run from the command line, if possible on a server, and in the background:

```
nohup sage find_counterexample.py 14 > out14.txt &          # large graphs
geng -c -D3 12 | sage find_counterexample.py > out12.txt &  # small graphs
```

With `nohup` the computation continues after you log out (alternatively use `screen` or `tmux`). We recommend the following conventions, so that the output can always be understood and nothing is lost if the computation is interrupted:

- at the start the script prints `----- START -----`;
- a script reading graphs from a file prints `----- END -----` after the last graph, so that we know that it has really finished; a heuristic search for large graphs usually runs in an infinite loop;
- from time to time (e.g. every 10^6 graphs, or less often for larger graphs) it prints the number of graphs checked so far;
- when a counterexample is found, it prints a clear message, the graph in graph6 format and its adjacency list (`G.to_dictionary()`), and it continues;
- after every `print` call `sys.stdout.flush()` (or use `print(..., flush=True)`), otherwise the output may stay in a buffer and be lost.

Before a long run, measure the time for smaller inputs and estimate the total time; split large jobs into parts with `res/mod` (Section 3) and run them in parallel on several cores.

8 Using AI tools

You may use AI tools (e.g. Claude, ChatGPT, Gemini, GitHub Copilot) for everything: brainstorming ideas, writing and debugging the code/algorithms, explaining a paper, setting up the correct ILP/ SAT problem. They are very useful, but they make mistakes, often with great confidence. Therefore:

- proofread and understand every AI output (eg. code) you use – at the defence you will have to explain it;
- test the code on small graphs and on graphs for which the answer is known, and compare two independent implementations (e.g. backtracking and ILP);
- never trust a “proof” or a “counterexample” produced by an AI tool without checking it yourself;
- say in the report what you used AI tools for.

9 What to report

The report should allow the reader to understand and repeat your experiments:

- the problem, the definitions and the conjecture you tested;
- your programs: the method (backtracking, ILP, heuristic, ...), how you tested their correctness, and a link to the code (GitHub);
- for the systematic search: the classes of graphs, the generator and its options, the largest order reached, the number of graphs checked and the running times;
- for the heuristic search: the search space, the neighbourhood, the objective function, the parameters, the number of runs, and the best results found;
- the results: tables, pictures of extremal graphs or counterexamples (in graph6 format as well), and your conclusions and new conjectures;
- a negative result is also a result: if you found no counterexample, report exactly what you checked (classes, orders, numbers of graphs, running times).

AI declaration

In the preparation of this document, the AI assistant Claude (Anthropic, https://claude.ai) was used, in particular for drafting the text and for writing and testing the code examples. All content was reviewed by the authors, who take full responsibility for it.

References

[1] C. Blum, A. Roli, *Metaheuristics in combinatorial optimization: Overview and conceptual comparison*, ACM Computing Surveys **35** (2003) 268–308.

[2] S. Luke, *Essentials of Metaheuristics*, Lulu, 2009; available at https://cs.gmu.edu/~sean/book/metaheuristics/.

[3] B. D. McKay, A. Piperno, *Practical graph isomorphism, II*, J. Symbolic Comput. **60** (2014) 94–112; see also https://pallini.di.uniroma1.it.

[4] The Sage Developers, *SageMath, the Sage Mathematics Software System*, https://www.sagemath.org.